

Turno

Manual técnico do projeto

Motor de agendamento e processamento de tarefas distribuídas em C# e .NET 10

Feito por Erik L. Pimenta

Versão 1.0.0 · Outubro de 2026 · Licença MIT

Sumário

1 **Visão geral** 4

Para que serve 4

O que o Turno faz 4

Ferramentas usadas 4

Identidade visual 5

2 **Conceitos** 7

Ciclo da tarefa 8

3 **Como rodar** 9

Primeira execução 9

O que sobe 9

Carga de demonstração 9

Sem Docker para as aplicações 10

Parar e limpar 10

4 **Guia do painel** 11

Chave de acesso 11

Visão geral 11

Tarefas 12

Detalhes da tarefa 13

Recorrentes 14

Workers 15

Fila de erros 16

Tema escuro 17

5 **Escrevendo handlers** 18

Estrutura 18

Registro 19

Enfileirando 19

6 **Garantias de execução** 20

Uma execução por tentativa 20

Servidores que caem 20

Encerramento gracioso 20

Rotinas com um único líder 20

7 **Resiliência** 21

Ajustes por tipo 21

8 **Escala e operação** 22

Mais workers 22

Filas e vagas 22

Redis com quórum 22

Azure Service Bus 22

Saúde e logs 22

Limpeza automática 23

9 **API** 24

Agendamento recorrente 25

Erros 25

10 **Configuração** 26

Variáveis do Docker 26

11 **Testes e automação** 28

Testes de unidade 28

Testes distribuídos 28

Integração contínua 28

Commits diários 28

12 **Solução de problemas** 29

Capítulo 1

Visão geral

O Turno é um motor de tarefas em segundo plano no estilo Hangfire. Uma API recebe as tarefas, vários workers idênticos dividem as mesmas filas e um painel mostra em tempo real o que está na fila, executando, com sucesso ou com falha. Cada tarefa roda uma única vez por tentativa, mesmo com mensagens duplicadas, workers que caem no meio do processo e reinícios do broker.

Para que serve

Trabalho pesado não deve rodar dentro da requisição do usuário: gerar relatórios, processar arquivos, sincronizar com parceiros, enviar notificações em massa. O Turno tira esse trabalho do caminho, distribui entre quantas máquinas forem necessárias e cuida do que dá errado: novas tentativas, disjuntor, tempo limite, fila de erros e recuperação de workers que somem.

O repositório entrega a estrutura, a arquitetura, a documentação e a ferramenta pronta para rodar. Quem usa configura as conexões, escreve os próprios handlers e decide quantos workers subir.

O que o Turno faz

- **Fila distribuída:** workers clonados consomem as mesmas filas do RabbitMQ ou do Azure Service Bus pelo MassTransit.
- **Uma execução por vez:** lock Redlock no Redis antes de começar e troca atômica de estado no PostgreSQL com token de despacho e lease.
- **Recuperação:** se um worker some, a tarefa volta para a fila sozinha e a tentativa fica registrada como worker perdido.
- **Resiliência com Polly:** nova tentativa imediata para falhas passageiras, disjuntor por tipo, tempo limite e backoff exponencial com jitter.
- **Fila de erros:** tarefas que esgotam as tentativas vão para a tabela `dead_letters` e para a fila `turno-dead-letter` do broker.
- **Agendamento:** imediato, com atraso, em data marcada ou recorrente por cron com fuso horário.
- **Chave de concorrência, idempotência, cancelamento e progresso** em todas as tarefas.
- **Painel em Blazor WebAssembly** com atualização por SignalR, tema claro e escuro e layout para celular.

Ferramentas usadas

Ferramenta	Versão	Papel no projeto
C# e .NET	.NET 10	Linguagem e plataforma de todos os projetos
Worker Service	.NET 10	Hospedagem dos workers, serviços em segundo plano e encerramento gracioso
ASP.NET Core	.NET 10	API REST com Minimal APIs, Problem Details e hospedagem do painel

Ferramenta	Versão	Papel no projeto
MassTransit	8.5.10	Mensageria, filas por worker, outbox transacional e fila de erros
RabbitMQ	4 (imagem Docker)	Broker padrão
Azure Service Bus	opcional	Broker alternativo, trocado só por configuração
Redis	7 (imagem Docker)	Locks distribuídos
RedLock.net	2.3.2	Algoritmo Redlock com quórum sobre o StackExchange.Redis 2.13.17
Polly	8.8.0	Nova tentativa, disjuntor e tempo limite por tipo de tarefa
PostgreSQL	17 (imagem Docker)	Estado das tarefas, histórico e outbox
Entity Framework Core	10.0.12	Acesso a dados e migrações, com o provedor Npgsql 10.0.3
Cronos	0.13.0	Expressões cron com segundos e fuso horário
Blazor WebAssembly	.NET 10	Painel que roda no navegador
SignalR	.NET 10	Atualização do painel em tempo real
OpenAPI e Scalar	.NET 10 e 2.18.0	Especificação e documentação interativa da API em `/docs`
xUnit	2.9.3	Testes de unidade e testes distribuídos
Docker Compose	v2	Ambiente completo com um comando
GitHub Actions		Integração contínua e commits diários de atividade

Identidade visual

O painel usa a mesma paleta das outras aplicações do autor: fundo cinza bem claro, superfícies brancas, bordas finas e um único tom de destaque, o verde-pinho, na navegação, nas barras de progresso, na seleção e no foco do teclado. A marca é sempre em preto e branco.

- #F6F7F7 — Fundo das páginas
- #FFFFFF — Painéis e tabelas
- #E2E5E6 — Bordas e divisórias
- #5F666B — Textos de apoio
- #15181B — Texto e botões
- #1F5C4C — Destaque e foco

Estado	Cor	Quando aparece
Executando	`#15181B`	Um worker está rodando a tarefa agora
Sucesso	`#1F6B45`	A tarefa terminou bem
Falha	`#A03A32`	Tentativas esgotadas ou erro permanente
Agendada	`#8A5A12`	Espera um horário: atraso, data marcada ou nova tentativa
Na fila	`#959B9F`	Pronta, aguardando uma vaga livre
Cancelada	`#C6CACC`	Cancelada antes ou durante a execução

As cores de estado aparecem só nos marcadores; o texto continua em tinta neutra. As fontes Red Hat Text e Red Hat Mono vêm junto com o projeto, sem CDN.

Capítulo 2

Conceitos

Os termos abaixo aparecem no painel, na API e no restante deste manual.

Termo	Significado
Tarefa	Um pedido de trabalho: tipo, fila, payload em JSON, limite de tentativas e tempo limite.
Tipo	Nome do handler que executa a tarefa, como `report.generate`.
Fila	Canal do broker por onde a tarefa chega aos workers. Cada worker escolhe quais filas atende e com quantas vagas.
Worker	Instância do `Turno.Worker`. Todos são iguais e podem ser adicionados ou removidos a qualquer momento.
Vaga	Uma execução simultânea dentro de um worker. A soma das vagas é a capacidade do cluster.
Tentativa	Cada execução de uma tarefa. Falhas passageiras geram novas tentativas até o limite.
Token de despacho	Identificador novo gerado sempre que a tarefa entra na fila. Só a mensagem com o token atual inicia a tarefa.
Lease	Prazo de posse da tarefa por um worker, renovado a cada 10 s. Se vence, outro worker pode assumir.
Chave de concorrência	Texto livre: tarefas com a mesma chave nunca rodam ao mesmo tempo.
Fila de erros	Lugar das tarefas que esgotaram as tentativas ou tiveram erro permanente.
Agendamento recorrente	Tarefa criada automaticamente por uma expressão cron.

Ciclo da tarefa

De	Para	Quando
criação	Agendada	Com atraso ou data futura
criação	Na fila	Para executar agora
Agendada	Na fila	O horário chegou
Na fila	Executando	Um worker assumiu a tarefa
Executando	Sucesso	O handler terminou sem erro
Executando	Agendada	Falhou e ainda há tentativas; disjuntor aberto; worker encerrado
Executando	Na fila	O worker parou de responder e o lease venceu
Executando	Falha	Tentativas esgotadas ou `PermanentJobException`
Executando	Cancelada	Cancelamento pedido durante a execução
Falha ou Cancelada	Na fila	Reenfileirada pelo painel ou pela API

Capítulo 3

Como rodar

O caminho mais curto é o Docker: um comando sobe banco, Redis, RabbitMQ, a API com o painel e três workers.

Primeira execução

1. Instale o **Docker** (Docker Desktop no Windows e no macOS).
2. Baixe o repositório e abra a pasta.
3. No Windows, dê dois cliques em `iniciar.cmd`. No Linux e no macOS, rode `./iniciar.sh`.
4. O script cria o `.env` a partir do `.env.example`, gera senhas aleatórias para PostgreSQL, Redis e RabbitMQ e uma chave de acesso para a API.
5. Quando a API responde, o navegador abre em **http://localhost:8080** e a chave aparece no terminal.

Dados sensíveis. O `.env` fica só na sua máquina e está no `.gitignore`. O repositório não traz senhas, chaves nem endereços reais.

O que sobe

Serviço	Endereço	Função
`api`	http://localhost:8080	API REST, painel e hub SignalR
`worker` × 3	sem porta	Workers que consomem as filas
`postgres`	127.0.0.1:5432	Estado das tarefas e outbox
`redis`	127.0.0.1:6379	Locks distribuídos
`rabbitmq`	http://localhost:15672	Broker e painel do RabbitMQ

Todas as portas ficam presas a `127.0.0.1`. As portas e a quantidade de workers podem ser trocadas no `.env` (`TURNO_PORT`, `TURNO_WORKERS` e as portas de cada serviço).

Carga de demonstração

Os quatro handlers de exemplo simulam relatórios, arquivos, integrações instáveis e notificações. Para enviar um lote misto:

```
./scripts/demo.sh 60
powershell -File scripts\demo.ps1 -Count 60
```

O lote inclui tarefas imediatas, atrasadas, com falha aleatória e uma que sempre falha, para mostrar novas tentativas, disjuntor e fila de erros no painel.

Sem Docker para as aplicações

Com o **.NET SDK 10** instalado, suba só a infraestrutura e rode API e workers pelo `dotnet`:

```
docker compose up -d postgres redis rabbitmq
```

Guarde as conexões com `dotnet user-secrets`, em `src/Turno.Api` e em `src/Turno.Worker`, para que nada fique no repositório:

```
dotnet user-secrets set "ConnectionStrings:Turno"
"Host=localhost;Database=turno;Username=turno;Password=<senha>"
dotnet user-secrets set "Turno:Messaging:RabbitMq:Username" "turno"
dotnet user-secrets set "Turno:Messaging:RabbitMq:Password" "<senha>"
dotnet user-secrets set "Turno:Redis:Connections:0" "localhost:6379,password=<senha>"
```

```
dotnet run --project src/Turno.Api
dotnet run --project src/Turno.Worker
dotnet run --project src/Turno.Worker
```

Cada `dotnet run` do worker é mais uma instância no cluster. A API aplica as migrações; os workers esperam o banco ficar pronto.

Parar e limpar

```
docker compose down
docker compose down -v
```

O primeiro comando para tudo e mantém os dados. O segundo apaga também os volumes do banco, do Redis e do RabbitMQ.

Capítulo 4

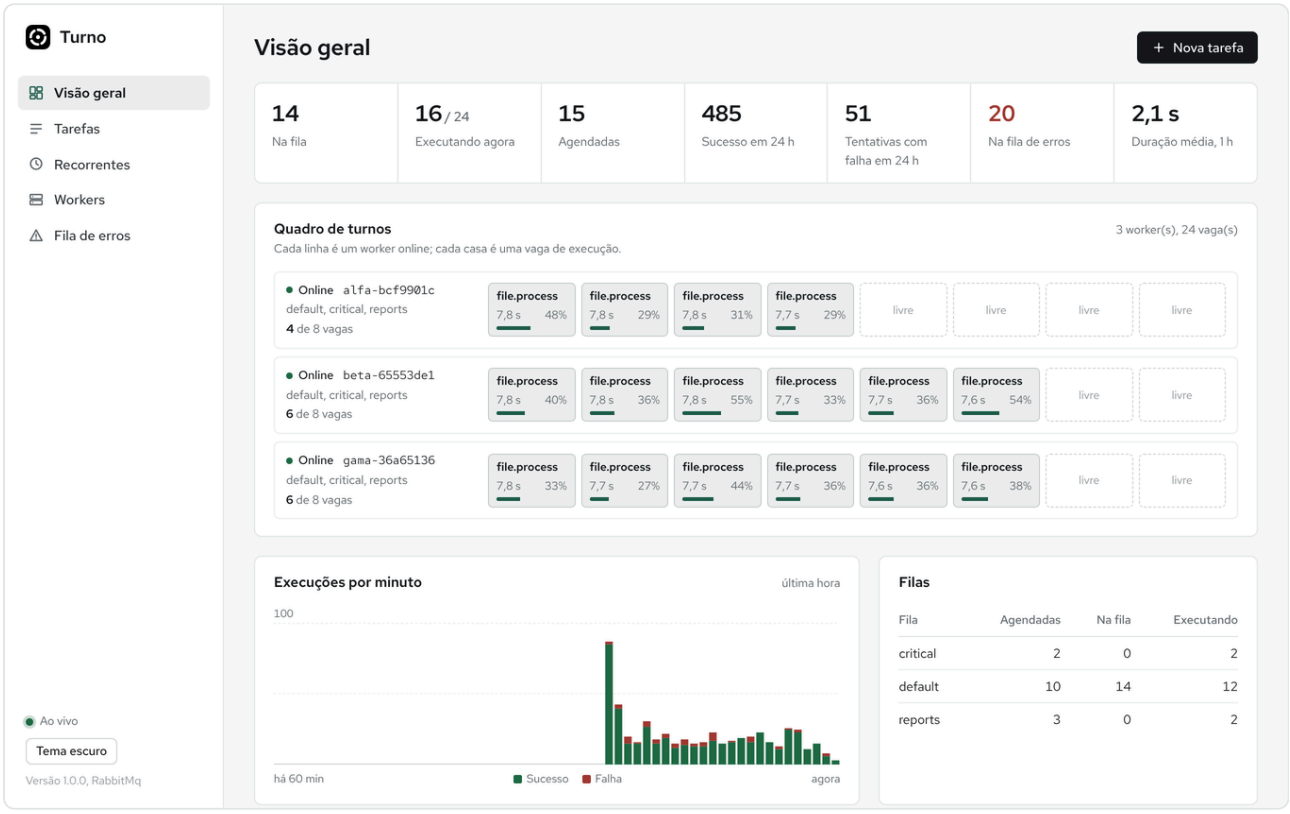
Guia do painel

O painel abre na visão geral e é atualizado em tempo real pelo SignalR. O indicador no rodapé da barra lateral mostra se a conexão está ao vivo.

Chave de acesso

Quando `Turno:Api:ApiKey` está definida, o painel pede a chave na primeira visita. Ela fica só na memória da aba e é enviada no cabeçalho `X-Turno-Key`. Sem chave configurada, o painel abre direto.

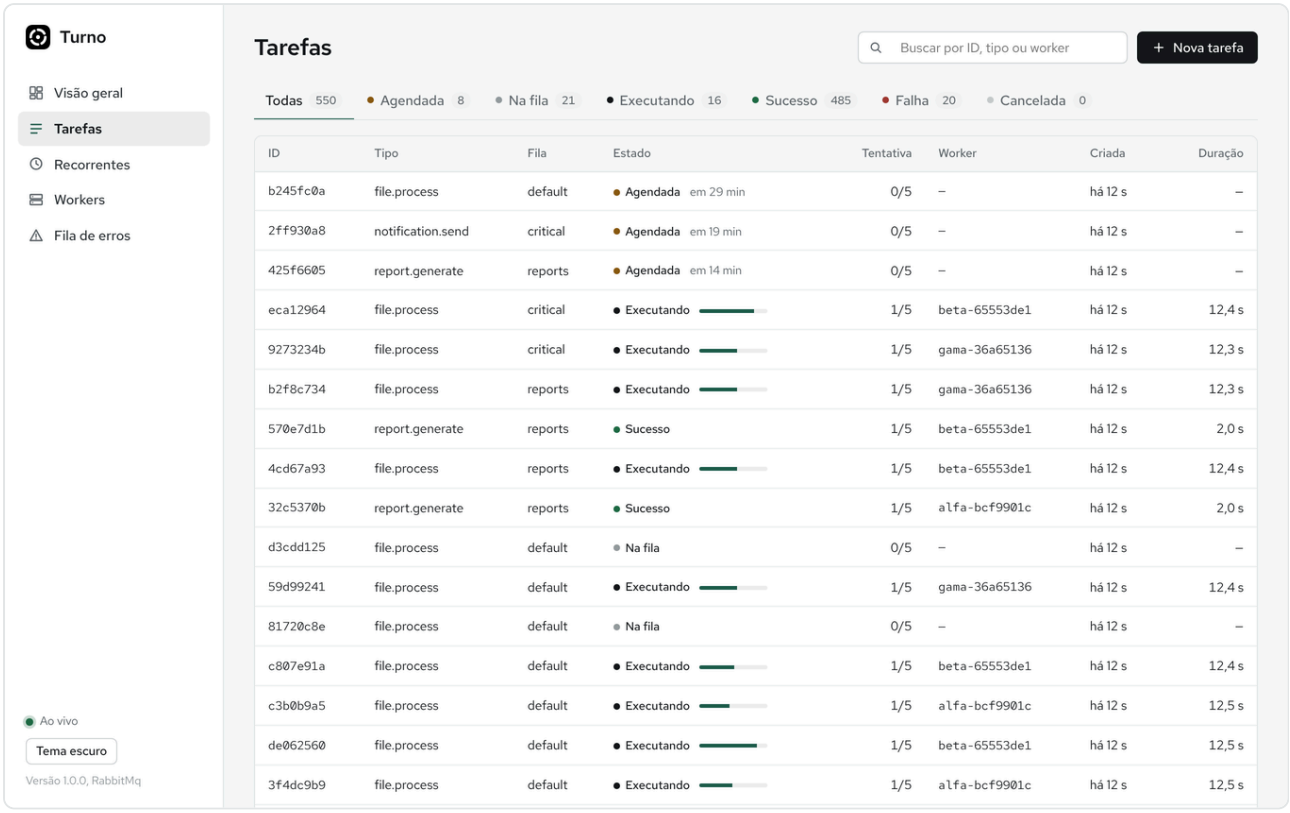
Visão geral



Visão geral com o quadro de turnos e as execuções da última hora

- **Faixa de indicadores:** na fila, executando agora sobre a capacidade total, agendadas, sucessos e tentativas com falha em 24 h, fila de erros e duração média da última hora.
- **Quadro de turnos:** cada linha é um worker online e cada casa uma vaga. Casas ocupadas mostram o tipo da tarefa, o tempo decorrido e o progresso; um clique abre os detalhes.
- **Execuções por minuto:** barras empilhadas de sucesso e falha na última hora.
- **Filas e falhas recentes:** tarefas pendentes por fila e as últimas entradas da fila de erros.

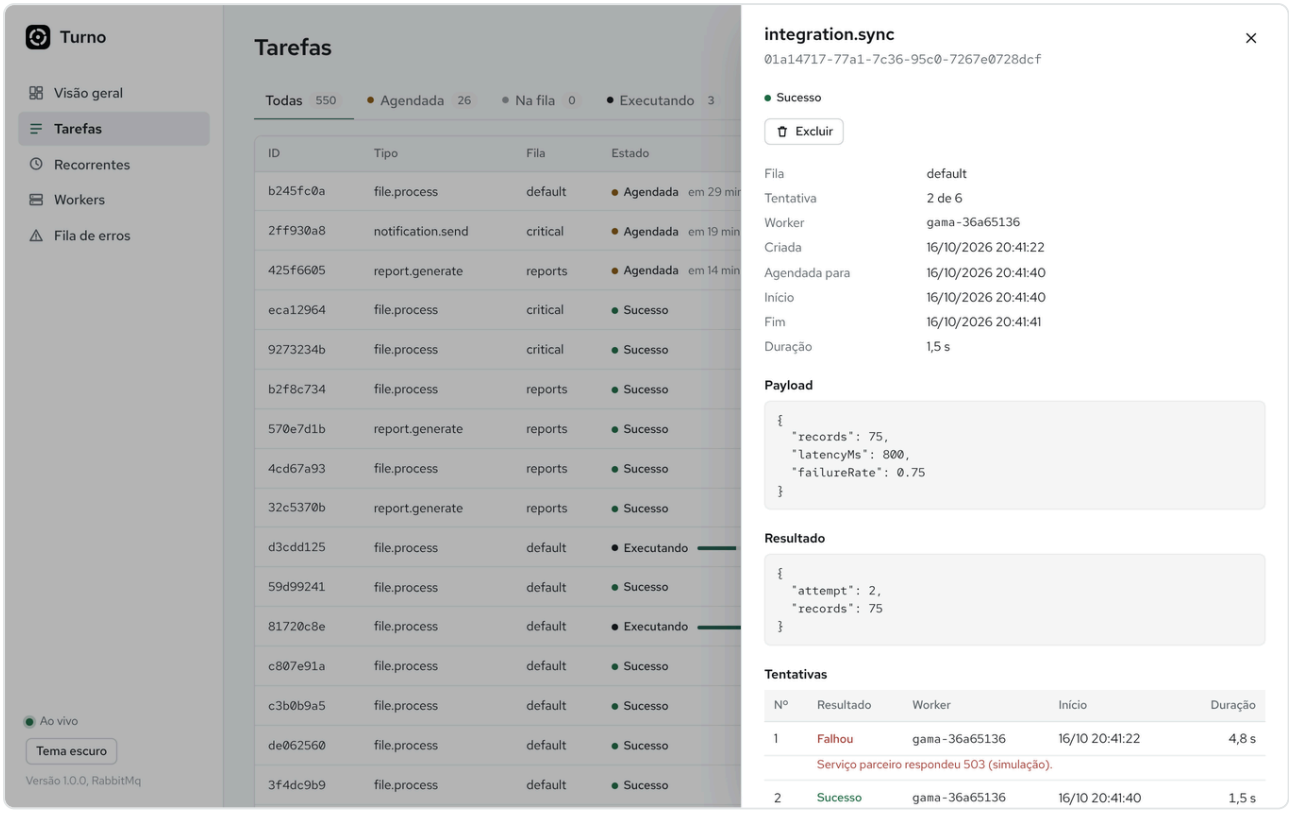
Tarefas



Lista de tarefas com filtro por estado e busca

Lista paginada com filtro por estado, busca por tipo, ID ou worker e colunas de fila, tentativa, worker, criação e duração. O botão **Nova tarefa** abre um formulário com tipo (com os tipos anunciados pelos workers), fila, payload em JSON, quando executar (agora, daqui a alguns segundos ou em data e hora), máximo de tentativas, tempo limite e chave de concorrência.

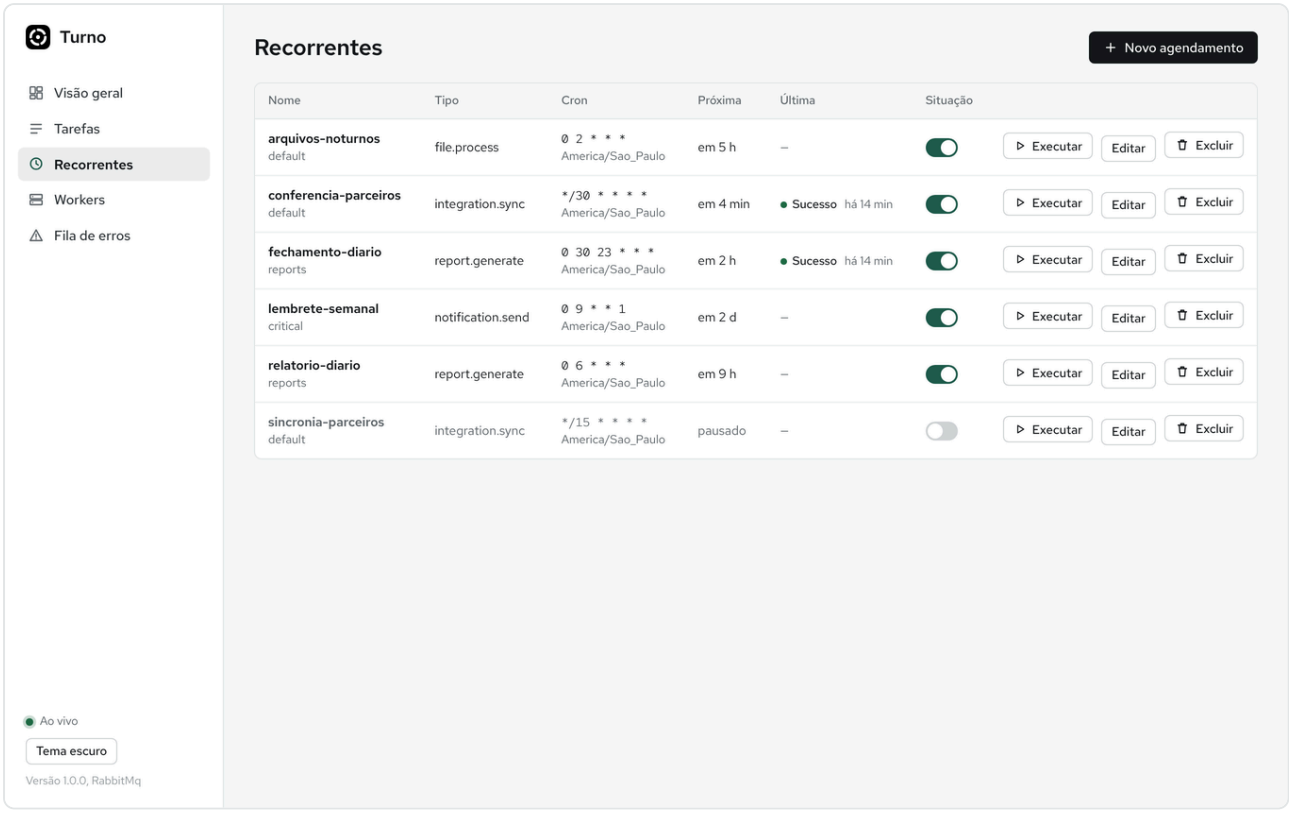
Detalhes da tarefa



Detalhes com tentativas, linha do tempo, payload e resultado

O painel lateral mostra o estado, as tentativas com worker, duração e erro, a linha do tempo de mudanças de estado, o payload e o resultado. Dali é possível cancelar, reenfileirar ou excluir, conforme o estado.

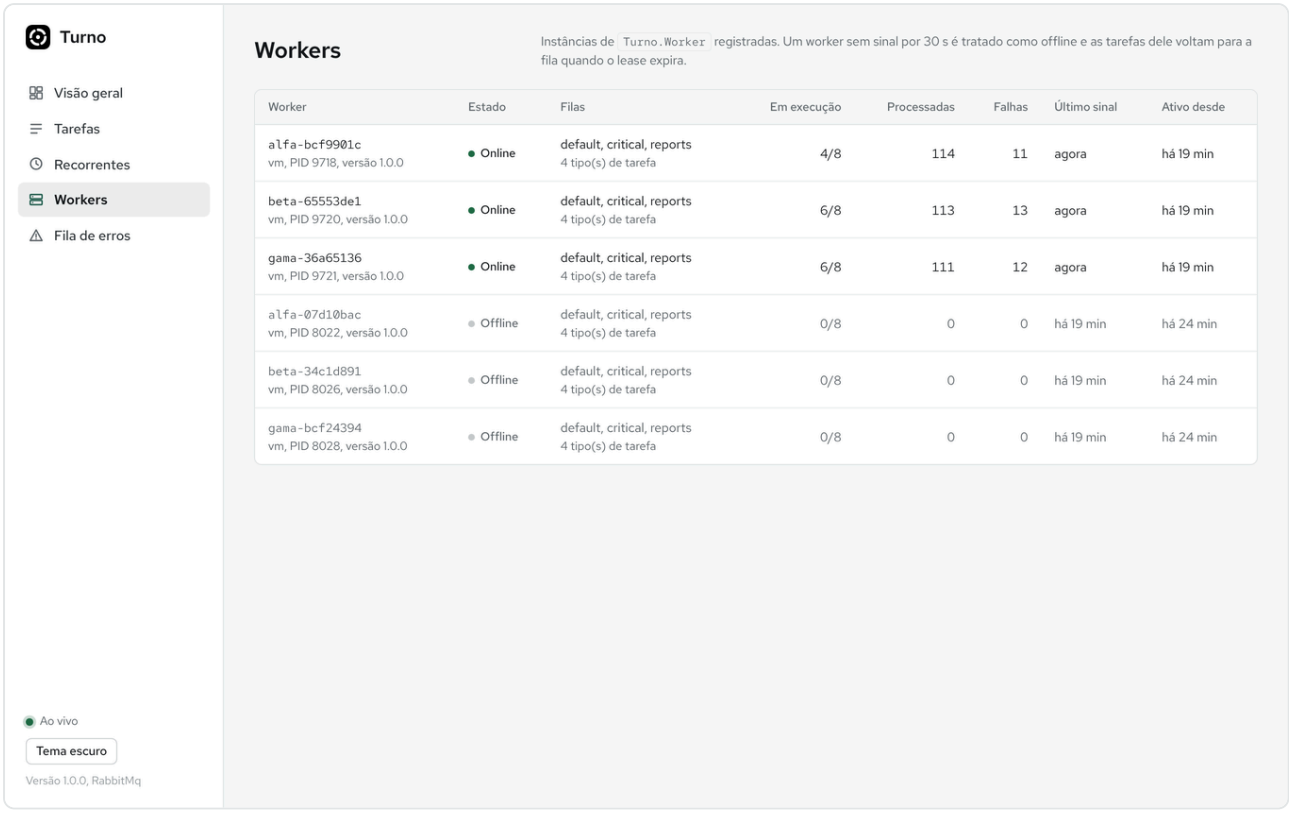
Recorrentes



Agendamentos recorrentes com próxima execução

Cria e edita agendamentos por cron (5 ou 6 campos) com fuso horário, fila, payload e se pode haver sobreposição. Cada linha pode ser disparada na hora, pausada, reativada ou excluída. Agendamentos também podem ser declarados em `Turno:RecurringJobs` no appsettings.

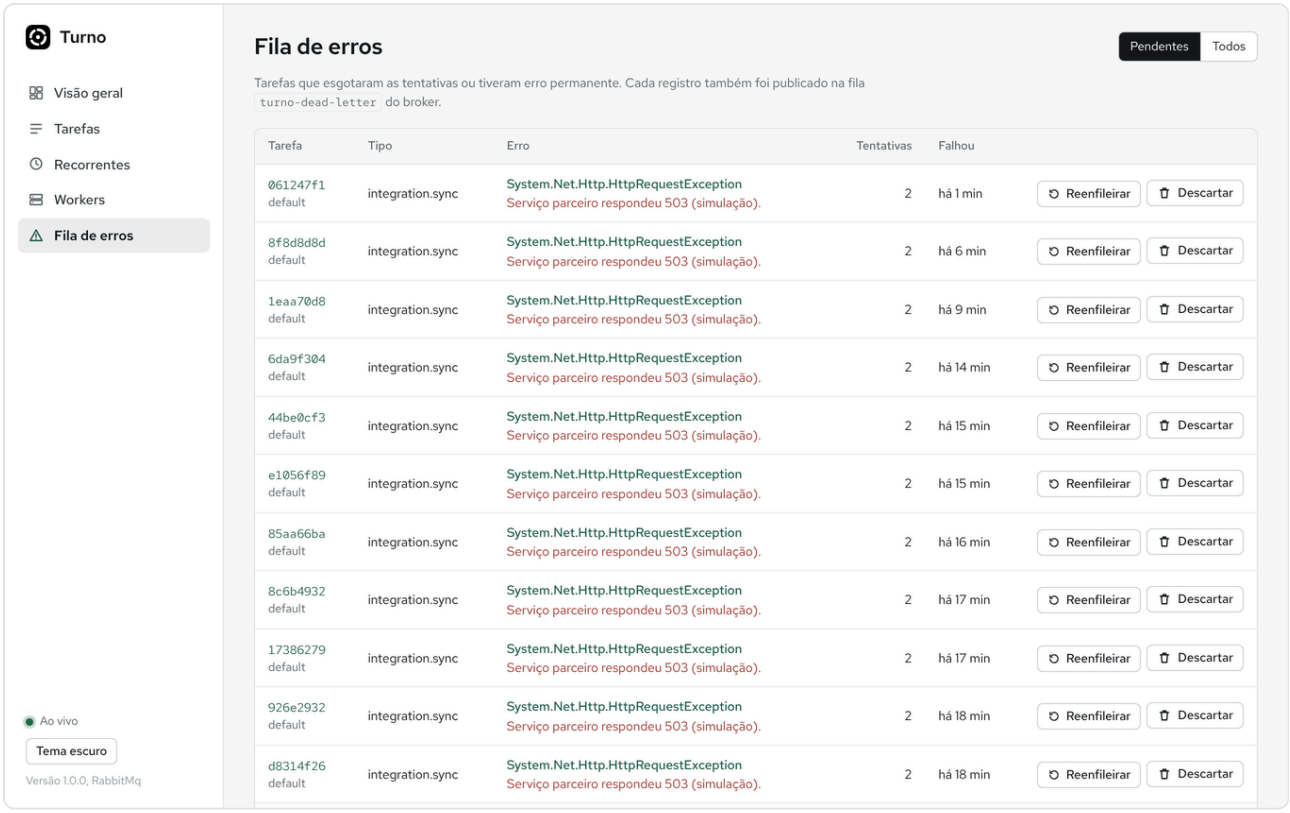
Workers



Workers registrados com filas, carga e último sinal de vida

Todas as instâncias registradas, com filas, vagas em uso, tarefas processadas e com falha, horário de início e último sinal de vida. Um worker que para de enviar sinal por 30 s aparece como offline e sai da lista depois de um dia.

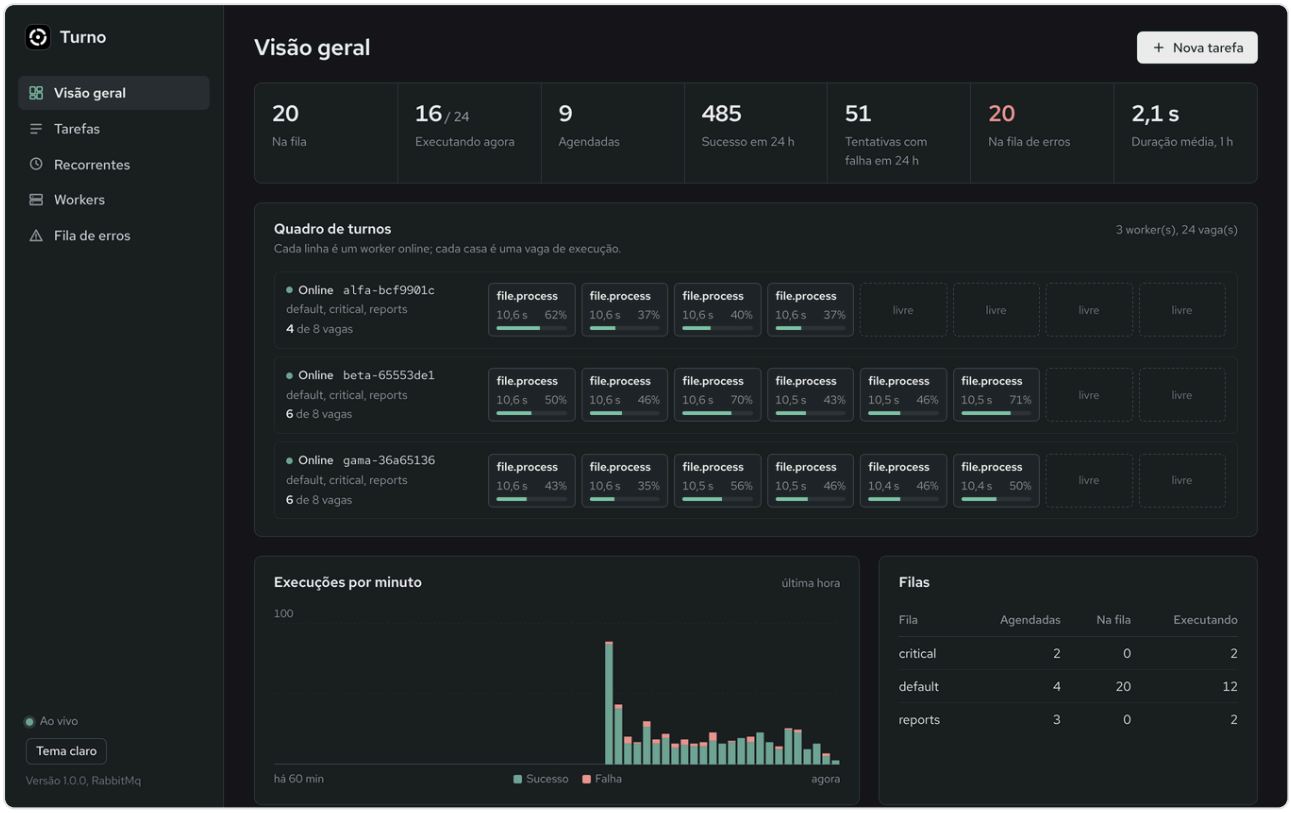
Fila de erros



Fila de erros com o erro de cada tarefa

Tarefas que esgotaram as tentativas ou tiveram erro permanente, com tipo do erro, mensagem e stack trace. **Reenfileirar** devolve a tarefa original à fila com novas tentativas; **Descartar** remove o registro.

Tema escuro



Visão geral no tema escuro

O botão na barra lateral alterna entre claro e escuro; a escolha fica guardada no navegador. Sem escolha, o painel segue o tema do sistema.

Capítulo 5

Escrevendo handlers

Um handler é uma classe que executa um tipo de tarefa. O Turno cuida de fila, lock, tentativas, progresso e cancelamento; o handler só faz o trabalho.

Estrutura

```
using Turno.Core.Handlers;

namespace MinhaEmpresa.Tarefas;

public sealed record GerarBoletosPayload(int Lote, string Competencia);

[JobType("boletos.gerar")]
public sealed class GerarBoletosHandler(IBoletoService boletos) :
JobHandler<GerarBoletosPayload>
{
    protected override async Task ExecuteAsync(GerarBoletosPayload payload, JobContext
context, CancellationToken cancellationToken)
    {
        if (payload.Lote <= 0)
        {
            throw new PermanentJobException("O lote precisa ser maior que zero.");
        }

        var total = await boletos.ContarAsync(payload.Lote, cancellationToken);
        for (var i = 0; i < total; i++)
        {
            await boletos.GerarAsync(payload.Lote, i, cancellationToken);
            await context.ReportProgressAsync((i + 1) * 100 / total, cancellationToken);
        }

        context.SetResult(new { total });
    }
}
```

Recurso	Como usar
Nome do tipo	`[JobType("...")]` com letras minúsculas, números, ponto, hífen ou sublinhado
Payload	Chega tipado e desserializado do JSON em camelCase; payload inválido vira erro permanente
Progresso	`context.ReportProgressAsync(0 a 100)`, enviado ao painel no máximo uma vez por segundo
Resultado	`context.SetResult(objeto)`, gravado em JSON (até 32 KB)
Cancelamento	O `CancellationToken` dispara em cancelamento, tempo limite, perda do lease e encerramento do worker
Erro passageiro	Qualquer exceção comum: o Polly tenta de novo e, depois, a tarefa volta com backoff

Recurso	Como usar
Erro permanente	`PermanentJobException`: vai direto para a fila de erros, sem novas tentativas
Dependências	Injetadas pelo construtor, com escopo próprio por execução

Registro

```
builder.Services.AddJobHandlersFromAssembly(typeof(GerarBoletosHandler).Assembly);
```

Coloque essa linha no `Program.cs` do worker. Os tipos registrados são anunciados pelo worker e aparecem no formulário de nova tarefa. Os handlers de exemplo podem ser desligados com `Turno:Samples:Enabled=false`.

Enfileirando

```
curl -X POST http://localhost:8080/api/jobs \
  -H "Content-Type: application/json" \
  -H "X-Turno-Key: <sua chave>" \
  -H "Idempotency-Key: boletos-2026-10-lote-7" \
  -d '{"type":"boletos.gerar","queue":"default","payload":{"lote":7,"competencia":"2026-10"}}'
```

Campo	Uso
`type`	Obrigatório. Tipo do handler
`queue`	Fila; `default` quando omitida
`payload`	Qualquer JSON, até 256 KB
`runAt` ou `delaySeconds`	Executa numa data e hora ou depois de um atraso
`maxAttempts`	Limite de tentativas (1 a 100); padrão da configuração quando omitido
`timeoutSeconds`	Tempo limite da execução
`idempotencyKey`	Evita duplicatas; também aceito no cabeçalho `Idempotency-Key`
`concurrencyKey`	Impede execuções simultâneas com a mesma chave

Capítulo 6

Garantias de execução

Como o Turno impede que duas instâncias rodem a mesma tarefa e o que acontece quando um servidor cai no meio do processo.

Uma execução por tentativa

1. **Token de despacho.** Toda vez que a tarefa entra na fila recebe um `dispatch_id` novo, que vai na mensagem. Reenvios, duplicatas e mensagens antigas carregam um token vencido e são descartados.
2. **Redlock.** O worker pega o lock `job:{id}` no Redis antes de tocar no banco. Com várias instâncias de Redis, o lock exige quórum. Se o lock está ocupado, a tarefa é adiada no banco e a mensagem é confirmada, sem perder trabalho.
3. **Troca atômica.** A passagem para executando é um único `UPDATE ... WHERE state = 'Enqueued' AND dispatch_id = @token`. Só uma instância consegue a linha.
4. **Lease com cerca.** O término só é gravado se o worker ainda for o dono (`worker_id`, `dispatch_id` e estado iguais). Um worker que travou e voltou não sobrescreve quem assumiu.
5. **Outbox transacional.** A mensagem para o broker é gravada na mesma transação da tarefa e entregue depois do commit. Se o banco recusou, nada é publicado.

Servidores que caem

Cada execução renova o lease a cada 10 s. O serviço de recuperação procura leases vencidos (60 s mais 15 s de tolerância), marca a tentativa como **worker perdido** e devolve a tarefa à fila com um token novo. Se o worker antigo voltar, a cerca descarta o resultado dele.

Encerramento gracioso

Ao receber o sinal de parada, o worker deixa de pegar tarefas e espera até 30 s pelas que estão rodando. As que não terminam são canceladas e voltam à fila sem contar tentativa. No Docker, o `stop_grace_period` é de 45 s para caber esse prazo.

Rotinas com um único líder

O agendador (tarefas agendadas e recorrentes), a recuperação e a limpeza rodam em todos os workers, mas a cada ciclo só um deles, eleito por um lock Redlock, executa a rotina. Dá para ligar quantos workers quiser sem duplicar esse trabalho.

Capítulo 7

Resiliência

Cada tipo de tarefa tem o próprio pipeline do Polly, montado na primeira execução e reaproveitado depois.

Camada	Ferramenta	Padrão
Tempo limite	Polly `Timeout`	10 min, ajustável por tipo ou por tarefa
Nova tentativa imediata	Polly `Retry`	2 novas tentativas a partir de 500 ms, só para erros passageiros
Disjuntor	Polly `CircuitBreaker`	Abre com 50% de falhas em 30 s (mínimo de 10 execuções) e fica aberto 30 s; as tarefas são adiadas sem contar tentativa
Novas tentativas da tarefa	Agendador do Turno	Até 5 tentativas, com espera de 15 s, 30 s, 60 s... até 15 min e 20% de jitter
Fila de erros	`dead_letters` e `turno-dead-letter`	Recebe a tarefa quando as tentativas acabam ou em erro permanente
Mensagens envenenadas	MassTransit `_error`	Mensagens que não conseguem nem ser processadas

Ajustes por tipo

```
"Turno": {
  "Resilience": {
    "DefaultMaxAttempts": 5,
    "BackoffBase": "00:00:15",
    "BackoffMax": "00:15:00",
    "Types": {
      "report.generate": { "Timeout": "00:30:00" },
      "integration.sync": {
        "ImmediateRetries": 3,
        "MaxAttempts": 6,
        "CircuitBreaker": { "FailureRatio": 0.3, "BreakDuration": "00:01:00" }
      }
    }
  }
}
```

Dica. Lance `PermanentJobException` para erros que nunca vão se resolver sozinhos (dados inválidos, registro inexistente). Assim a tarefa não gasta tentativas nem segura vagas.

Capítulo 8

Escala e operação

Os workers não guardam estado: qualquer um pode morrer, reiniciar ou ser adicionado a qualquer momento.

Mais workers

```
docker compose up -d --scale worker=8
```

Fora do Docker, basta iniciar mais processos `Turno.Worker` apontando para o mesmo banco, Redis e broker, em quantas máquinas quiser.

Filas e vagas

```
"Turno": {
  "Worker": {
    "Queues": [
      { "Name": "default", "Concurrency": 4 },
      { "Name": "critical", "Concurrency": 2 },
      { "Name": "reports", "Concurrency": 2 }
    ]
  }
}
```

Workers diferentes podem atender filas diferentes: por exemplo, máquinas maiores só com `reports`. A fila no broker se chama `turno-<fila>`; o prefixo muda em `Turno:Messaging:QueuePrefix`.

Redis com quórum

Para desenvolvimento, uma conexão basta. Em produção, use 3 ou 5 instâncias independentes de Redis em `Turno:Redis:Connections`: o lock só é concedido com a maioria, como no Redlock original. Sem quórum, o worker adia a tarefa em vez de arriscar.

Azure Service Bus

```
Turno__Messaging__Transport=AzureServiceBus
Turno__Messaging__AzureServiceBus__ConnectionString=<connection string do namespace>
```

Nada mais muda: filas, fila de erros e eventos do painel funcionam igual.

Saúde e logs

Rota	O que verifica
`/health/live`	Processo da API no ar

Rota	O que verifica
`/health/ready`	PostgreSQL, quórum do Redis e conexão com o broker

Os logs saem no console em uma linha por evento, em português, com o ID da tarefa e o worker. No Docker: `docker compose logs -f worker`.

Limpeza automática

Tarefas com sucesso e canceladas são removidas depois de 7 dias, as com falha depois de 30 e os workers offline depois de 1 dia. Os prazos ficam em `Turno:Retention`.

Capítulo 9

API

Documentação interativa em http://localhost:8080/docs e especificação OpenAPI em /openapi/v1.json.

Com `Turno:Api:ApiKey` definida, todas as rotas, menos `/api/info` e `/health`, exigem o cabeçalho `X-Turno-Key`.

Método	Rota	Função
POST	`/api/jobs`	Enfileira ou agenda uma tarefa
POST	`/api/jobs/batch`	Enfileira até 1000 tarefas
GET	`/api/jobs`	Lista com filtros (`state`, `type`, `queue`, `search`) e paginação
GET	`/api/jobs/{id}`	Detalhes, tentativas e linha do tempo
POST	`/api/jobs/{id}/cancel`	Cancela uma tarefa agendada, na fila ou em execução
POST	`/api/jobs/{id}/requeue`	Devolve à fila uma tarefa com falha ou cancelada
DELETE	`/api/jobs/{id}`	Exclui uma tarefa finalizada
GET	`/api/stats`	Indicadores, vazão por minuto e filas
GET	`/api/workers`	Workers registrados
GET	`/api/job-types`	Tipos anunciados pelos workers online
GET · PUT	`/api/recurring-jobs`	Lista; cria ou atualiza pelo nome
POST	`/api/recurring-jobs/{id}/trigger`	Dispara agora
POST	`/api/recurring-jobs/{id}/enable` · `/disable`	Ativa ou pausa
DELETE	`/api/recurring-jobs/{id}`	Exclui o agendamento
GET	`/api/dead-letters`	Fila de erros (`pending=true` para só as pendentes)
POST	`/api/dead-letters/{id}/requeue`	Reenfileira a tarefa original
DELETE	`/api/dead-letters/{id}`	Descarta o registro
WS	`/hubs/jobs`	Hub SignalR com `JobsChanged` e `WorkersChanged`

Agendamento recorrente

```
curl -X PUT http://localhost:8080/api/recurring-jobs \
  -H "Content-Type: application/json" \
  -H "X-Turno-Key: <sua chave>" \
  -d '{"name":"fechamento-diario","type":"report.generate","queue":"reports","cron":"0 30 23 *
* *","timeZone":"America/Sao_Paulo","payload":{"rows":250000}}'
```

Com 6 campos, o primeiro é o segundo. Sem `allowConcurrent`, uma nova execução não é criada enquanto a anterior ainda estiver agendada, na fila ou executando.

Erros

Os erros seguem o padrão Problem Details (RFC 9457), com mensagens em português e os campos inválidos em `errors`:

```
{ "title": "Dados inválidos para a tarefa.", "status": 400, "errors": { "type": ["Informe o
tipo da tarefa com letras minúsculas, números, ponto, hífen ou sublinhado (até 200
caracteres)."] } }
```

Capítulo 10

Configuração

As opções ficam no appsettings.json da API e do worker e podem ser sobrescritas por variáveis de ambiente, trocando ":" por "__".

Seção	Principais opções	Padrão
`ConnectionStrings:Turno`	Conexão com o PostgreSQL	obrigatória
`Turno:Database`	`ApplyMigrations`	ligado na API, desligado nos workers do Docker
`Turno:Messaging`	`Transport`, `QueuePrefix`, credenciais do RabbitMQ ou connection string do Azure Service Bus	`RabbitMq`, `turno`
`Turno:Redis`	`Connections`, `KeyPrefix`	`localhost:6379`, `turno`
`Turno:Worker`	`Name`, `Queues`, `LeaseDuration`, `LeaseRenewalInterval`, `LockExpiry`, `HeartbeatInterval`, `ShutdownTimeout`	nome da máquina; 60 s, 10 s, 30 s, 5 s, 30 s
`Turno:Scheduler`	`Interval`, `BatchSize`	2 s, 200
`Turno:Recovery`	`Interval`, `LeaseGracePeriod`, `RedispatchEnqueuedAfter`	10 s, 15 s, desligado
`Turno:Retention`	Prazos de guarda e de worker offline	7 d, 7 d, 30 d, 1 d
`Turno:Resilience`	Tentativas, backoff, tempo limite, disjuntor e `Types`	veja o capítulo Resiliência
`Turno:Api`	`ApiKey`, `AllowedOrigins`, `EnableApiReference`	sem chave, sem CORS, docs ligadas
`Turno:RecurringJobs`	Agendamentos declarados na configuração	2 exemplos
`Turno:Samples`	`Enabled`	ligado

Variáveis do Docker

Variável	Uso
`TURNO_PORT`	Porta do painel e da API (8080)
`TURNO_WORKERS`	Quantidade inicial de workers (3)
`TURNO_API_KEY`	Chave de acesso; vazia desliga a exigência
`TURNO_POSTGRES_*`	Banco, usuário, senha e porta do PostgreSQL

Variável	Uso
`TURNO_RABBITMQ_*`	Usuário, senha e portas do RabbitMQ
`TURNO_REDIS_*`	Senha e porta do Redis
`TZ`	Fuso dos containers (`America/Sao_Paulo`)

Capítulo 11

Testes e automação

Duas suítes de testes, integração contínua e o workflow de atividade diária.

Testes de unidade

```
dotnet test tests/Turno.Tests
```

Cobrem máquina de estados, cron, backoff, validação das requisições, handlers, pipeline do Polly e o modelo do banco, incluindo a checagem de que não há mudança no modelo sem migração.

Testes distribuídos

```
docker compose up -d postgres redis rabbitmq
export TURNO_TEST_POSTGRES="Host=localhost;Username=turno;Password=<senha>;Database=postgres"
export TURNO_TEST_REDIS="localhost:6379,password=<senha>"
export TURNO_TEST_RABBITMQ="amqp://turno:<senha>@localhost:5672"
dotnet test tests/Turno.IntegrationTests
```

Cada teste sobe um cluster com vários workers contra serviços reais, num banco e num prefixo de filas próprios, e prova: 60 tarefas em 3 workers rodam uma vez cada; duplicatas e mensagens antigas são ignoradas; um worker travado tem a tarefa recuperada e o resultado descartado; tentativas esgotadas vão para a fila de erros; erro permanente pula as tentativas; a chave de concorrência impede sobreposição; cancelamento, encerramento gracioso, agendamento, recorrência e idempotência funcionam.

Integração contínua

O workflow `ci.yml` compila a solução, roda as duas suítes com PostgreSQL, Redis e RabbitMQ como serviços e gera as duas imagens Docker a cada push na `main`.

Commits diários

Turno	Horário principal	Reserva	Brasília
1º turno	04:23 UTC	05:41 UTC	01:23 e 02:41
2º turno	07:23 UTC	08:41 UTC	04:23 e 05:41

O workflow `diario.yml` registra a atividade em `.github/atividade.txt`. Os horários ficam na janela de menor carga do GitHub Actions, longe da virada de hora e sem coincidir com os outros repositórios do autor; o horário reserva só faz commit se o principal não rodou. Crie os secrets `GIT_NAME` e `GIT_EMAIL` em **Settings › Secrets and variables › Actions**.

Capítulo 12

Solução de problemas

Sintoma	Causa provável	O que fazer
O painel não abre	Containers ainda subindo	`docker compose ps` e `docker compose logs api`
O painel pede a chave	`TURNO_API_KEY` definida	Use a chave mostrada pelo script ou a do `.env`
"Sem conexão em tempo real"	API reiniciando ou proxy sem WebSocket	O painel reconecta sozinho; libere WebSocket no proxy
Tarefas ficam na fila	Nenhum worker atende aquela fila ou não há vagas	Confira as filas em Workers e o `Turno:Worker:Queues`
Tarefas adiadas com "disjuntor aberto"	Muitas falhas seguidas do mesmo tipo	Corrija a dependência; o disjuntor fecha sozinho
Workers esperando o banco	Migrações ainda não aplicadas	Suba a API, que aplica as migrações, ou ligue `ApplyMigrations` em um worker
Redis sem quórum	Instâncias de Redis fora do ar	Verifique `/health/ready`; as tarefas são adiadas até o quórum voltar
Porta ocupada	Outro serviço usa a mesma porta	Troque a porta no `.env` e rode o script de novo

Recomeçar do zero. `docker compose down -v` apaga banco, filas e locks. Depois, rode o script de início de novo.